



Создание gRPC клиента и сервера на Java Spring

В современной архитектуре микросервисов приложениям необходимо эффективно общаться друг с другом. gRPC - это современный высокопроизводительный фреймворк от Google для удаленного вызова процедур (RPC), который является ключевым инструментом в построении таких систем. В отличие от традиционного REST API, gRPC использует Protocol Buffers для сериализации данных, что обеспечивает меньший размер передаваемых сообщений и более высокую скорость. Освоение gRPC позволит вам создавать быстрые, строго типизированные и масштабируемые межсервисные взаимодействия, что является неотъемлемым навыком для любого Java-разработчика.

2. Теория

Что такое gRPC?

gRPC (Google Remote Procedure Call) - это высокопроизводительный фреймворк с открытым исходным кодом, который позволяет клиенту напрямую вызывать методы в серверном приложении, как если бы это был локальный объект. gRPC используется для создания быстрых и эффективных API в распределенных системах и микросервисах. Он работает поверх HTTP/2, что позволяет использовать мультиплексирование, потоковую передачу и другие современные возможности протокола.

Что такое Protocol Buffers?

Protocol Buffers (Protobuf) - это механизм сериализации структурированных данных, разработанный Google. Он используется в gRPC для определения интерфейса сервиса и структуры сообщений. Protobuf позволяет описать структуру данных в специальном файле с расширением .proto. Из этого файла автоматически генерируется исходный код для выбранного языка программирования (в нашем случае — Java), который можно использовать для легкой и быстрой работы со структурированными данными.

Синтаксис .proto файла

Файл .proto является сердцем gRPC-сервиса. В нем описываются сервисы (контракты), их методы и сообщения (структуры данных).

Пример синтаксиса:

```
// Указывает версию синтаксиса  
syntax = "proto3";
```



```
// Опции для генерации Java кода
option java_multiple_files = true;
option java_package = "grpc.demo";

// Определение сервиса, который будет предоставлять методы
service UserService {
    // Определение RPC-метода
    // Принимает CreateUserRequest и возвращает CreateUserResponse
    rpc CreateUser (CreateUserRequest) returns (CreateUserResponse) {}
}

// Определение сообщения (структуры данных)
message User {
    // Каждое поле имеет тип, имя и уникальный номер
    string user_id = 1;
    string name = 2;
    string email = 3;
}

message CreateUserRequest {
    string name = 1;
    string email = 2;
}

message CreateUserResponse {
    User user = 1;
}
```

3. Практический разбор

Создадим систему управления пользователями и их достижениями. Сервер будет хранить информацию, а клиент будет запрашивать ее. Мы создадим два метода - один для создания пользователя, а другой для получения списка его достижений.



Определение контракта в user-service.proto

Создадим файл user-service.proto в директории src/main/proto. Этот файл будет общим для сервера и клиента (пока поступим просто, потом уже сделаем отдельный проект).

```
syntax = "proto3";
```

```
// Опции для генерации Java-классов в указанном пакете.
```

```
option java_multiple_files = true;
```

```
option java_package = "grpc.demo";
```

```
option java_outer_classname = "UserServiceProto";
```

```
// Определение сервиса для управления пользователями
```

```
service UserService {
```

```
    // Метод для создания нового пользователя
```

```
    rpc CreateUser(CreateUserRequest) returns (CreateUserResponse) {}
```

```
    // Метод для получения достижений пользователя по его ID
```

```
    rpc      GetUserBadges(GetUserBadgesRequest)      returns  
(GetUserBadgesResponse) {}  
}
```

```
// Сообщение-объект, ачивка-достижение
```

```
message Badge {
```

```
    string badge_id = 1;
```

```
    string name = 2;
```

```
    string description = 3;
```

```
}
```

```
// Сообщение-объект, пользователь
```

```
message User {
```

```
    string user_id = 1;
```

```
    string name = 2;
```



```
}
```

```
// Запрос на создание пользователя
```

```
message CreateUserRequest {  
    string name = 1;  
}
```

```
// Ответ после создания пользователя
```

```
message CreateUserResponse {  
    User user = 1;  
}
```

```
// Запрос на получение достижений
```

```
message GetUserBadgesRequest {  
    string user_id = 1;  
}
```

```
// Ответ, содержащий список достижений
```

```
message GetUserBadgesResponse {  
    // 'repeated' означает, что это будет список  
    repeated Badge badges = 1;  
}
```

Создание gRPC Сервера

Настройка проекта Maven

Создайте новый Spring Boot проект grpc-server. В файл pom.xml добавьте зависимости для gRPC сервера и плагин для генерации кода из .proto файла.

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>grpc.demo</groupId>  
    <artifactId>grpc-server</artifactId>
```



```
<version>1.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  <protobuf.version>3.25.3</protobuf.version>
  <protobuf-plugin.version>0.6.1</protobuf-plugin.version>
  <grpc.version>1.66.0</grpc.version>
</properties>

<dependencies>
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-stub</artifactId>
    <version>${grpc.version}</version>
  </dependency>
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-protobuf</artifactId>
    <version>${grpc.version}</version>
  </dependency>
  <dependency>
    <groupId>io.grpc</groupId>
    <artifactId>grpc-netty-shaded</artifactId>
    <version>${grpc.version}</version>
  </dependency>
  <dependency>
    <groupId>jakarta.annotation</groupId>
    <artifactId>jakarta.annotation-api</artifactId>
    <version>1.3.5</version>
    <optional>true</optional>
  </dependency>

  <dependency>
    <groupId>net.devh</groupId>
    <artifactId>grpc-server-spring-boot-
starter</artifactId>
    <version>3.1.0.RELEASE</version>
  </dependency>

</dependencies>

<build>
  <extensions>
    <extension>
      <groupId>kr.motd.maven</groupId>
      <artifactId>os-maven-plugin</artifactId>
```



```
        <version>1.7.0</version>
      </extension>
    </extensions>

    <plugins>
      <plugin>
        <groupId>org.xolstice.maven.plugins</groupId>
        <artifactId>protobuf-maven-plugin</artifactId>
        <version>${protobuf-plugin.version}</version>
        <configuration>

<protocArtifact>com.google.protobuf:protoc:${protobuf.version}:e
xe:${os.detected.classifier}</protocArtifact>
        <pluginId>grpc-java</pluginId>
        <pluginArtifact>io.grpc:protoc-gen-grpc-
java:${grpc.version}:exe:${os.detected.classifier}</pluginArtifa
ct>

        </configuration>
        <executions>
          <execution>
            <goals>
              <goal>compile</goal>
              <goal>compile-custom</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-
plugin</artifactId>
      </plugin>
    </plugins>
  </build>

</project>
```

Реализация сервиса

Создайте класс `UserServiceImpl`, который будет реализовывать логику нашего сервиса.

```
package grpc.demo;

import io.grpc.stub.StreamObserver;
import net.devh.boot.grpc.server.service.GrpcService;
import grpc.demo.*;
```



| gRPC

```
import java.util.List;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

@GrpcService // Аннотация, которая регистрирует этот класс как
gRPC сервис
public class UserServiceImpl extends
UserServiceGrpc.UserServiceImplBase {

    private final ConcurrentMap<String, User> users = new
ConcurrentHashMap<>();

    @Override
    public void createUser(CreateUserRequest request,
StreamObserver<CreateUserResponse> responseObserver) {
        // Создаем нового пользователя с уникальным ID
        User newUser = User.newBuilder()
            .setUserId(UUID.randomUUID().toString())
            .setName(request.getName())
            .build();
        users.put(newUser.getUserId(), newUser);

        System.out.println("Создан пользователь: " +
newUser.getName());

        // Создаем и отправляем ответ
        CreateUserResponse response =
CreateUserResponse.newBuilder().setUser(newUser).build();
        responseObserver.onNext(response);
        responseObserver.onCompleted();
    }

    @Override
    public void getUserBadges(GetUserBadgesRequest request,
StreamObserver<GetUserBadgesResponse> responseObserver) {
        String userId = request.getUserId();

        // Имитируем получение достижений для пользователя
        System.out.println("Запрошены достижения для
пользователя: " + userId);
        Badge badge1 = Badge.newBuilder()
            .setBadgeId("b1")
            .setName("Первый шаг")
            .setDescription("Успешно создан аккаунт")
            .build();
        Badge badge2 = Badge.newBuilder()
            .setBadgeId("b2")
            .setName("Любопытный")
            .setDescription("Запросил свои достижения")
    }
```

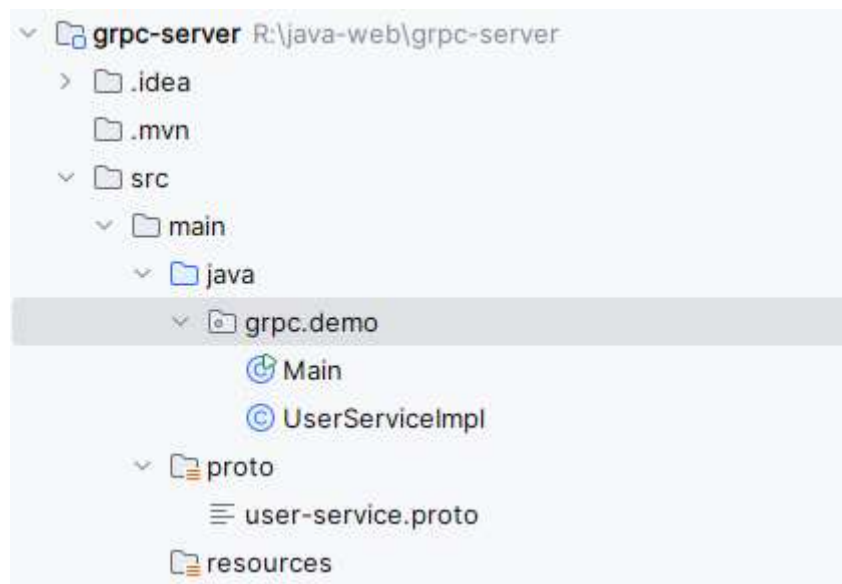


```
.build();

// Создаем и отправляем ответ со списком достижений
GetUserBadgesResponse response =
GetUserBadgesResponse.newBuilder()
    .addAllBadges(List.of(badge1, badge2))
    .build();

responseObserver.onNext(response);
responseObserver.onCompleted();
}
}
```

Вот так должна быть выглядеть структура проекта:



Maven

Жмем на  в , если структура у вас была верная, должно все сработать,

УБЕДИТЕСЬ, что пакет у вас единый как в proto, так в pom, так и в коде!

Main класс

```
package grpc.demo;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication()
public class Main {
    public static void main(String[] args) {
```



```
        SpringApplication.run(Main.class, args);
    }
}
```

Запуск сервера

Запустите главный класс вашего Spring Boot приложения. По умолчанию, gRPC сервер стартует на порту 9090.

Создание gRPC клиента

Настройка проекта Maven

Создайте **второй, отдельный** Spring Boot проект. Настройка pom.xml будет почти такой же, как у сервера, но зависимость будет клиентская (позначил красным).

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>grpc.demo</groupId>
    <artifactId>grpc-client</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
        <protobuf.version>3.25.3</protobuf.version>
        <protobuf-plugin.version>0.6.1</protobuf-plugin.version>
        <grpc.version>1.66.0</grpc.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>io.grpc</groupId>
            <artifactId>grpc-stub</artifactId>
            <version>${grpc.version}</version>
        </dependency>
        <dependency>
            <groupId>io.grpc</groupId>
            <artifactId>grpc-protobuf</artifactId>
            <version>${grpc.version}</version>
        </dependency>
        <dependency>
            <groupId>io.grpc</groupId>
            <artifactId>grpc-netty-shaded</artifactId>
```



| gRPC

```
        <version>${grpc.version}</version>
      </dependency>
      <dependency>
        <groupId>jakarta.annotation</groupId>
        <artifactId>jakarta.annotation-api</artifactId>
        <version>1.3.5</version>
        <optional>true</optional>
      </dependency>

      <dependency>
        <groupId>net.devh</groupId>
        <artifactId>grpc-client-spring-boot-
starter</artifactId>
        <version>3.1.0.RELEASE</version>
      </dependency>

    </dependencies>

    <build>
      <extensions>
        <extension>
          <groupId>kr.motd.maven</groupId>
          <artifactId>os-maven-plugin</artifactId>
          <version>1.7.0</version>
        </extension>
      </extensions>

      <plugins>
        <plugin>
          <groupId>org.xolstice.maven.plugins</groupId>
          <artifactId>protobuf-maven-plugin</artifactId>
          <version>${protobuf-plugin.version}</version>
          <configuration>

            <protocArtifact>com.google.protobuf:protoc:${protobuf.version}:e
xe:${os.detected.classifier}</protocArtifact>
            <pluginId>grpc-java</pluginId>
            <pluginArtifact>io.grpc:protoc-gen-grpc-
java:${grpc.version}:exe:${os.detected.classifier}</pluginArtifa
ct>

          </configuration>
          <executions>
            <execution>
              <goals>
                <goal>compile</goal>
                <goal>compile-custom</goal>
              </goals>
            </execution>
          </executions>
        </plugin>
      </plugins>
    </build>
  </parent>
</project>
```



```
        </executions>
    </plugin>
    <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-
plugin</artifactId>
    </plugin>
</plugins>
</build>

</project>
```

Важно - плагин для генерации кода (protobuf-maven-plugin) и файл user-service.proto должны быть и в клиентском проекте тоже.

Настройка соединения

В файле src/main/resources/application.properties укажите адрес сервера.

Указываем, что для клиента с именем "user-service" нужно подключаться по этому адресу.

```
grpc.client.user-service.address=static://localhost:9090
```

Используем plaintext, так как не настраивали шифрование.

```
grpc.client.user-service.negotiation-type=plaintext
```

Создание тестового компонента для вызова RPC

Создадим компонент, который при старте приложения вызовет методы на сервере.

```
package grpc.demo;

import io.grpc.StatusRuntimeException;
import net.devh.boot.grpc.client.inject.GrpcClient;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import grpc.demo.*;

@Component
public class GrpcClientRunner implements CommandLineRunner {

    // Внедряем заглушку (stub) для нашего gRPC сервиса
    // user-service - имя, которое мы задали в
    application.properties
    @GrpcClient("user-service")
    private UserServiceGrpc.UserServiceBlockingStub
userServiceStub;
```



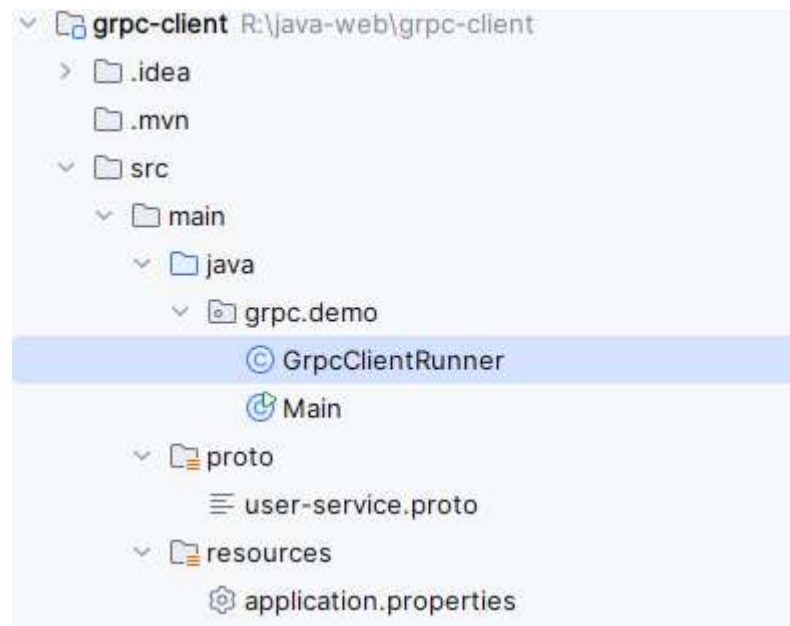
```
@Override
public void run(String... args) {
    System.out.println("--- Клиент gRPC запущен ---");
    try {
        // 1. Создаем пользователя
        System.out.println("--> Вызов CreateUser...");
        CreateUserRequest createRequest =
        CreateUserRequest.newBuilder().setName("Студент").build();
        CreateUserResponse createResponse =
        userServiceStub.createUser(createRequest);
        String newUserId =
        createResponse.getUser().getUserId();
        System.out.println("<-- Пользователь успешно создан
        с ID: " + newUserId);

        // 2. Получаем его достижения
        System.out.println("\n--> Вызов GetUserBadges для
        пользователя " + newUserId + "...");
        GetUserBadgesRequest badgesRequest =
        GetUserBadgesRequest.newBuilder().setUserId(newUserId).build();
        GetUserBadgesResponse badgesResponse =
        userServiceStub.getUserBadges(badgesRequest);
        System.out.println("<-- Получены достижения:");
        for (Badge badge : badgesResponse.getBadgesList()) {
            System.out.println("    - " + badge.getName() +
            " (" + badge.getDescription() + ")");
        }

    } catch (StatusRuntimeException e) {
        System.err.println("!!! Ошибка при вызове gRPC: " +
        e.getStatus());
    }
    System.out.println("--- Клиент gRPC завершил работу ---
    ");
}
}
```



Структура проекта



Maven

Жмем на  в , если структура у вас была верная, должно все сработать,

УБЕДИТЕСЬ, что пакет у вас единый как в proto, так в pom, так и в коде!

Запуск клиента

Сначала убедитесь, что сервер запущен. Затем запустите главный класс клиентского Spring Boot приложения. Вы должны увидеть в консоли клиента лог вызовов и ответы от сервера.

```
--- Клиент gRPC запущен ---
--> Вызов CreateUser...
<-- Пользователь успешно создан с ID: 06268b67-efa0-47a1-b829-df737d978928

--> Вызов GetUserBadges для пользователя 06268b67-efa0-47a1-b829-df737d978928...
<-- Получены достижения:
    - Первый шаг (Успешно создан аккаунт)
    - Любопытный (Запросил свои достижения)
--- Клиент gRPC завершил работу ---
```

4. Задания для самостоятельной работы

1. Расширение модели пользователя:

1. Добавьте в сообщение User в файле user-service.proto новое поле `string email = 3;`



1. Обновите .proto файл в обоих проектах.
 2. Пересоберите оба проекта (mvn clean install), чтобы сгенерировать новый Java-код.
 3. Измените метод createUser на сервере так, чтобы он принимал email в CreateUserRequest и устанавливал его для нового пользователя.
 4. Обновите клиентский код, чтобы он отправлял email при создании пользователя и выводил его в консоль после создания.
2. Добавление метода обновления пользователя:
1. Добавьте в сервис UserService новый метод gpc
UpdateUser(UpdateUserRequest) returns (UpdateUserResponse) {}.
 1. Определите сообщения UpdateUserRequest (должен содержать user_id и новые данные, например name) и UpdateUserResponse (должен содержать обновленный объект User).
 2. Реализуйте логику метода на сервере: он должен находить пользователя по user_id и обновлять его имя.
 3. Вызовите новый метод с клиента после создания пользователя.
3. Обработка ошибки «Пользователь не найден»:
1. Реализуйте обработку случая, когда клиент запрашивает достижения для несуществующего user_id.
 1. В методе getUserBadges на сервере добавьте проверку, существует ли пользователь с запрошенным user_id.
 2. Если пользователя нет, выбросьте исключение Status.NOT_FOUND.withDescription("Пользователь не найден").asRuntimeException().
 3. В клиентском коде (GrpcClientRunner) оберните вызов getUserBadges в блок try-catch и обработайте StatusRuntimeException, выводя в консоль сообщение об ошибке, если статус ответа NOT_FOUND.

Всегда следите за синхронизацией .proto файлов между клиентом и сервером. Любое расхождение в контракте приведет к ошибкам во время выполнения. В реальных



| gRPC

проектах для этого создают отдельный interface-модуль, который используется как зависимость и клиентом, и сервером.